

Python Data Structures

Lists, Tuples, Sets, Dictionaries, Indexing, Slicing, and Comprehensions

1. Introduction

Real-world programs rarely work with single, isolated values — they work with collections of data: a list of students, a set of unique tags, a dictionary mapping usernames to profiles. Python provides four essential built-in data structures for organizing collections of data: lists, tuples, sets, and dictionaries. Each has different characteristics around ordering, mutability, and uniqueness, and choosing the right one for a task is a key programming skill.

This module covers how to create and use each structure, how indexing and slicing let you access portions of ordered data, the most common built-in methods for each type, the powerful list comprehension syntax, and practical guidance on when to choose one structure over another.

2. Learning Objectives

- Create and manipulate lists, tuples, sets, and dictionaries.
- Use indexing and slicing to access elements and sub-sequences.
- Apply common built-in methods for each data structure.
- Write list comprehensions to build lists concisely.
- Perform common dictionary operations: adding, updating, removing, and iterating over key-value pairs.
- Explain when to prefer a list vs a tuple, and a set vs a list.

3. Concept Explanations

3.1 Lists

A list is an ordered, mutable (changeable) collection that can hold items of any type, including mixed types. Lists are created with square brackets.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")           # add to the end
fruits.insert(1, "kiwi")       # insert at index 1
fruits.remove("banana")        # remove by value
fruits.pop()                   # remove and return last item
print(len(fruits))             # number of items
print(fruits)
```

3.2 Tuples

A tuple is an ordered, **immutable** collection — once created, its contents cannot be changed. Tuples are created with parentheses (or simply commas) and are often used for fixed collections of related values,

such as coordinates.

```
point = (3, 7)
coordinates = 10, 20, 30      # parentheses are optional
x, y = point                  # tuple unpacking
print(x, y)                   # 3 7
# point[0] = 5                 # TypeError: tuples are immutable
```

3.3 Sets

A set is an unordered collection of **unique** items. Sets automatically remove duplicates and provide fast membership testing along with mathematical operations like union and intersection.

```
colors = {"red", "green", "blue", "red"}
print(colors)                  # duplicates removed: {'red', 'green', 'blue'}
colors.add("yellow")
colors.discard("green")

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)                   # union: {1, 2, 3, 4}
print(a & b)                   # intersection: {2, 3}
print(a - b)                   # difference: {1}
```

3.4 Dictionaries

A dictionary stores data as key-value pairs, allowing fast lookup of a value by its unique key. Keys must be immutable (strings, numbers, or tuples); values can be anything.

```
student = {"name": "Alice", "age": 22, "grade": "A"}
print(student["name"])         # Alice
student["age"] = 23            # update a value
student["email"] = "a@x.com"   # add a new key
del student["grade"]           # remove a key
print(student.get("phone", "N/A")) # safe lookup with default
for key, value in student.items():
    print(key, "->", value)
```

3.5 Indexing and Slicing

Indexing accesses a single element by position (starting at 0; negative indices count from the end). Slicing extracts a sub-sequence using [start:stop:step], where stop is exclusive.

```
letters = ['a', 'b', 'c', 'd', 'e']
print(letters[0])              # 'a'
print(letters[-1])             # 'e'
print(letters[1:4])            # ['b', 'c', 'd']
print(letters[:3])             # ['a', 'b', 'c']
print(letters[:2])            # ['a', 'c', 'e']
print(letters[::-1])          # ['e', 'd', 'c', 'b', 'a'] (reversed)
```

3.6 Common Methods Summary

Structure	Common Methods
-----------	----------------

list	append(), insert(), remove(), pop(), sort(), reverse(), index(), count()
tuple	count(), index() (few methods, since tuples are immutable)
set	add(), remove(), discard(), union(), intersection(), difference()
dict	get(), keys(), values(), items(), update(), pop(), setdefault()

3.7 List Comprehensions

A list comprehension builds a new list in a single, readable expression, often replacing a multi-line for loop with append() calls.

```
squares = [x ** 2 for x in range(1, 6)]
print(squares)                # [1, 4, 9, 16, 25]
```

```
evens = [x for x in range(20) if x % 2 == 0]
print(evens)
```

```
# Equivalent long form:
evens2 = []
for x in range(20):
    if x % 2 == 0:
        evens2.append(x)
```

3.8 List vs Tuple, Set vs List

Comparison	Key Difference	When to Use
List vs Tuple	Lists are mutable; tuples are immutable	Use a tuple for fixed data that should not change (e.g. RGB values);
Set vs List	Sets are unordered and store only unique elements	Use sets when you need fast membership tests or must guarantee uniqueness

4. Syntax Summary

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
my_set = {1, 2, 3}
my_dict = {"a": 1, "b": 2}

sequence[index]
sequence[start:stop:step]

[expression for item in iterable if condition]    # list comprehension
```

5. Code Examples

Example 1: Shopping Cart with a List

```
cart = []
cart.append("Bread")
cart.append("Milk")
cart.append("Eggs")
```

```
print(f"You have {len(cart)} items: {cart}")
```

Example 2: Word Frequency with a Dictionary

```
text = "the quick brown fox jumps over the lazy dog the fox runs"
words = text.split()
frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1
print(frequency)
```

Example 3: Removing Duplicates with a Set

```
raw_emails = ["a@x.com", "b@x.com", "a@x.com", "c@x.com", "b@x.com"]
unique_emails = list(set(raw_emails))
print(unique_emails)
```

6. Step-by-Step Explanation

Walking through Example 2 (Word Frequency) line by line:

- 1 `text.split()` breaks the string into a list of words using whitespace as the separator.
- 2 An empty dictionary `frequency` is created to hold each word's count.
- 3 For every word in the list, `frequency.get(word, 0)` looks up the current count, defaulting to 0 if the word has not been seen yet.
- 4 Adding 1 and reassigning updates (or creates) the entry for that word.
- 5 After the loop, `frequency` maps each unique word to how many times it appeared in the text.

7. Common Mistakes

- **Common Mistake:** trying to modify a tuple, e.g. `my_tuple[0] = 5`, which raises a `TypeError`.
- **Common Mistake:** using a list where a set would be more appropriate for uniqueness checks, causing slow, repeated 'in' checks on large data.
- **Common Mistake:** accessing a dictionary key that does not exist with square brackets, causing a `KeyError`, instead of using `.get()`.
- **Common Mistake:** assuming sets preserve insertion order — they do not, and iterating over one can produce items in any order.
- **Common Mistake:** confusing slicing bounds, forgetting that the stop index in a slice is exclusive.

8. Best Practices

- ✓ **Best Practice:** use tuples for fixed, related groups of values and lists for collections that change over time.
- ✓ **Best Practice:** use `.get()` with a default value instead of direct key access when a key might be missing.
- ✓ **Best Practice:** prefer list comprehensions over manual loops for simple transformations — they are more readable and often faster.

✓ **Best Practice:** use sets to efficiently deduplicate data or perform membership tests on large collections.

9. Practice Questions

- 1 What is the output of `[1, 2, 3, 4, 5][1:-1]`?
- 2 Why can a tuple be used as a dictionary key, but a list cannot?
- 3 Write a list comprehension that produces the squares of only the odd numbers from 1 to 10.
- 4 What does `my_dict.items()` return, and how is it typically used?
- 5 Given two sets `a` and `b`, what is the difference between `a - b` and `b - a`?

10. Mini Coding Exercises

- 1 Write a program that takes a list of numbers and returns a new list with only the even numbers, using a list comprehension.
- 2 Write a program that merges two dictionaries, where values from the second dictionary override the first on key conflicts.
- 3 Write a program that finds the common elements between two lists using sets.
- 4 Write a program that reverses a tuple without modifying the original.
- 5 Write a program that counts how many times each character appears in a string using a dictionary.

11. Interview / QC Questions

- 1 What is the time complexity of checking membership (`'in'`) in a list versus a set, and why?
- 2 Explain why dictionary keys must be hashable, and give an example of an unhashable type.
- 3 How would you efficiently remove duplicates from a list while preserving the original order?
- 4 What is the difference between shallow copy and deep copy when copying a list of lists?
- 5 Compare `dict.get(key)` and `dict[key]` in terms of error handling and typical use cases.

12. Summary

This module introduced Python's four core built-in data structures. Lists offer ordered, mutable collections ideal for data that changes over time; tuples offer ordered, immutable collections ideal for fixed groups of values; sets offer unordered, unique collections ideal for membership testing and deduplication; and dictionaries offer fast key-based lookup ideal for structured records. You also learned how indexing and slicing extract data from ordered sequences, how list comprehensions build lists concisely, and practical guidance for choosing the right structure for a given problem. These structures are the backbone of nearly every non-trivial Python program.

End of Python_Data_Structures — continue to Python_OOP.pdf for the next module.